

Runbook - Sincronização X, Y (mm) e ângulo visão-CLP/robô

Campo	Valor
Sistema	Realtec Vision Buddmeyer v2.0
Baseline	v2108
Público	Integração em campo (POC Curitiba; operação remota via AnyDesk)
Relacionado	TAG_CONTRACT.md v1.1 FEATURE_COORDINATE_MAPPING.md RUNBOOK_INTEGRACAO_CLP.md

Contrato do software (o que o Vision realmente envia)

O sistema de visão não implementa affine, homografia nem offset de origem. A transformação v1 é só escala, em coordinate/transform.py:

```
píxeis da imagem (origem canto superior esquerdo, X -> direita, Y ↓ baixo)
-> clamp ao ROI (se activo)
-> X_mm = X_px x mm/px
    Y_mm = Y_px x mm/px
-> ângulo [0, 180) no eixo X da imagem (horário no ecrã)
-> tags CIP: CENTROID_X, CENTROID_Y (REAL, mm), CENTROID_ANGLE (REAL, graus)
```

A sincronização com o referencial do robô faz-se por montagem da câmara + mm/px + correcção no programa do CLP (sinais, troca de eixos, translação X0/Y0, offset de ângulo).

O robô só deve usar X, Y e theta depois do handshake deste ciclo (VisionCtrl_DataSent / envio da FSM), não a partir do overlay contínuo.

Trabalhar sempre no POC (câmara e robô reais). AnyDesk opera a UI; régua e peça no tabuleiro são físicas, em Curitiba.

Etapa 0 - Condições e papéis

Parar o robô em modo seguro (manual / teach). Ninguém na zona sem o procedimento de segurança da célula.

Quem	Função
Visão	Overlay, tags CIP, valor `mm/px`
CLP / robô	Origem de mundo, sinais, offset, movimento
Operação	Régua e peça no plano de pick

Anotar antes de começar:

- IP do NX102
- Nomes das tags: CENTROID_X, CENTROID_Y, CENTROID_ANGLE (ver TAG_CONTRACT.md)
- Unidades: mm e graus

Câmara o mais perpendicular possível ao plano de pick (nadir). Inclinação gera erro que esta versão não corrige.

Etapa 1 - Dois referenciais (escrever num papel)

Visão (imagem)

- Origem: canto superior esquerdo do frame (não do ROI, salvo o efeito do clamp).
- +X: direita no ecrã.
- +Y: para baixo no ecrã (convenção de imagem).
- Ângulo 0°: eixo maior da embalagem paralelo ao +X da imagem.
- Sentido positivo: horário no ecrã.
- Intervalo: [0, 180) (embalagem retangular simétrica a 180°).

Robô / CLP

- Origem: a que o movimento usa (base, tabuleiro, user frame).
- +X / +Y: os do teach pendant.
- Ângulo da garra: convenção da ferramenta (0-360° ou ±180°).

Enquanto estes dois desenhos não estiverem no papel, não calibrar números.

Etapa 2 - ROI e plano de trabalho

1. Arrancar o Vision no POC (AnyDesk se remoto): USB da célula -> Iniciar.
2. Na aba Operação, definir o ROI como zona segura do contentor (não as paredes).
3. Salvar a configuração.
4. Com ROI ligado, o ponto enviado pode ser deslizado para dentro do retângulo ao longo do eixo da embalagem, para o robô não ir às paredes.
5. Todas as medições (régua, peça, marcas) no mesmo plano Z do pick.

Etapa 3 - Escala mm/px (régua)

Isto só resolve tamanho: quantos milímetros tem um pixel. Não resolve origem nem rotação de eixos.

1. Colocar régua rígida no plano de pick, visível, alinhada o mais possível com X ou Y da imagem.
2. Correr no POC a aplicação de calibração por régua (AnyDesk para a UI). Fórmula manual:

$$\text{mm/px} = \text{comprimento_real_mm} / \text{comprimento_em_pixels}$$

3. No Vision: Configuração -> Imagem -> Centroide (mm/px) -> gravar o valor -> Salvar.
4. Copiar o valor para config.yaml.poc no box PC.
5. Verificar no overlay: uma peça de 200 mm deve aparecer cerca de 200 mm na etiqueta. Dobro ou metade indica inversão px/mm.

O default 1,0 (1 px = 1 mm) é inválido na célula. Reiniciar a app após gravar, para o transform não ficar com o valor antigo em cache.

Etapa 4 - Origem e eixos (coração da sincronização)

O software faz $X_{mm} = X_{px} \times k$. O píxel (0,0) não é a origem do robô. Alinhar no CLP (ou no user frame):

$$\begin{aligned} X_{clp} &= s_x \cdot X_{visao} + X0 \\ Y_{clp} &= s_y \cdot Y_{visao} + Y0 \end{aligned}$$

- s_x, s_y : +1 ou -1 (e, se preciso, troca $X \leftrightarrow Y$).
- $X0, Y0$: translação em mm.

4.1 Marca física

Marcar 3 pontos no tabuleiro (P1, P2, P3), visíveis à câmara e acessíveis ao TCP, no plano de pick. Distâncias conhecidas.

4.2 Ler visão

Para cada ponto (marcador ou embalagem centrada na marca):

- Overlay: X, Y em mm (já com mm/px), ou
- Ciclo manual: autorizar envio e ler CENTROID_X / CENTROID_Y no Sysmac nesse ciclo.

Anotar: P1_visao, P2_visao, P3_visao.

4.3 Ler robô

Teach: TCP no mesmo ponto físico. Anotar P1_robô, P2_robô, P3_robô no user frame do pick.

4.4 Deduzir sinais e offset

- Se andar para a direita no ecrã e o X do robô diminuir $\rightarrow s_x = -1$.
- Se Y da visão cresce para baixo e o Y do robô cresce noutro sentido $\rightarrow$ provavelmente $s_y = -1$.
- Se X da visão acompanha Y do robô $\rightarrow$ trocar eixos no CLP.

Com P1:

$$\begin{aligned} X0 &= X_{robô} - s_x \cdot X_{visao} \\ Y0 &= Y_{robô} - s_y \cdot Y_{visao} \end{aligned}$$

Validar em P2 e P3.

- Erro de poucos mm, constante: escala ok; ajustar X0/Y0.
- Erro que cresce com a distância: câmara inclinada ou distorção. Esta v1 não corrige (affine/homography abertos). Reduzir o ROI ao centro ou nivelar a câmara.

Implementar $s_x, s_y, X0, Y0$ (e swap) no programa do CLP/robô, não no YAML do Vision.

Etapa 5 - Ângulo da embalagem

O Vision envia o eixo maior da máscara, [0, 180), referencial imagem.

1. Embalagem alinhada com +X da imagem (lado comprido horizontal no ecrã). Overlay $\sim 0^\circ$ (ou 180° ,

equivalente).

2. Rodar cerca de 45° no sentido horário no ecrã; o valor deve subir.
3. No robô, com a mesma peça, anotar o ângulo da ferramenta que pega bem.

No CLP, em geral:

```
theta_garra = theta_visao + theta_offset
```

Se o sentido for o contrário:

```
theta_garra = -theta_visao + theta_offset
```

Como o Vision só dá [0, 180), o CLP deve tratar theta e theta+180° como o mesmo objecto se a garra for simétrica. Testar as duas se a primeira colidir.

theta_offset calibra-se com uma peça bem pega; não misturar com o offset de X, Y.

Etapa 6 - Handshake (quando o robô pode confiar nos números)

Sincronizar valores não chega se o robô ler tags a meio do frame.

1. CLP: RobotCtrl_AuthorizeDetection = TRUE.
2. Visão em DETECTING; PickStabilizer (vários frames); envia um alvo.
3. Escritas: CENTROID_X, CENTROID_Y, CENTROID_ANGLE, OBJECT_AREA, PRODUCT_DETECTED, VisionCtrl_DataSent, VisionBusy.
4. CLP aplica s, X0, Y0, theta_offset e move.
5. ROBOT_ACK -> pick -> RobotStatus_PickComplete -> place -> RobotStatus_PlaceComplete.

Não interpolar o overlay “ao vivo” para o motion. Só o pacote deste ciclo.

Etapa 7 - Ensaio de aceitação

Repetir com peça parada (não em movimento na passadeira, se houver).

Teste	Critério sugerido
3 marcas P1-P3	Distância TCP – ponto físico dentro da tolerância de pick (definir na célula, ex. ±3 a 5 mm)
Peça no centro do ROI	X, Y no Sysmac iguais aos esperados após a fórmula do CLP
Peça perto do bordo do ROI (ainda válida)	Sem colisão com parede; clamp visível se o centróide saía
Ângulo 0° e ~45°	Garra alinhada à embalagem nas duas
Um ciclo manual completo	ACK -> pick -> place sem TIMEOUT

Diagnóstico rápido:

- X/Y ok e ângulo mal -> só etapa 5.
- Tudo proporcionalmente errado -> etapa 3 (mm/px).
- Um canto certo e o oposto errado -> geometria da câmara (etapas 0 e 1), não o mm/px.

Etapa 8 - Congelar e não perder no git pull

No POC, guardar:

- valor mm/px
- ROI (x, y, w, h)
- s_x, s_y, X0, Y0, theta_offset comentados no programa do CLP (fonte da verdade do robô)
- ficheiro config.yaml.poc

Um git pull do Vision não deve apagar isto. Offset de robô não vive no GitHub do portátil.

Limite desta versão

Com câmara a prumo, tabuleiro plano e offset no CLP, X, Y (mm) e ângulo ficam sincronizáveis para pick-and-place.

Se a câmara estiver oblíqua ou o tabuleiro for grande com distorção, a escala única não chega: nivelar a óptica ou, mais tarde, affine/homography no Vision. Até lá, ROI pequeno no centro e a tabela P1-P3 são o contrato de campo.

Ordem curta

1. Nivelar câmara
2. ROI
3. Régua mm/px
4. Três pontos visão vs teach
5. Sinais e offset no CLP
6. Offset de ângulo
7. Um ciclo handshake
8. Gravar números no POC e no programa do robô

© Realtec - Buddmeyer Vision System v2.0 (baseline v2108)